EXPERIMENT NO: 07

Aim: Simulate the Distributed Mutual Exclusion.

Theory:

Mutual exclusion in distributed system

Mutual exclusion is a concurrency control property which is introduced to prevent race conditions. It is the requirement that a process cannot enter its critical section while another concurrent process is currently present or executing in its critical section i.e. only one process is allowed to execute the critical section at any given instance of time.

Mutual exclusion in single computer system Vs. distributed system: In single computer system, memory and other resources are shared between different processes. The status of shared resources and the status of users is easily available in the shared memory so with the help of shared variable (For example: Semaphores) mutual exclusion problem can be easily solved. In Distributed systems, we neither have shared memory nor a common physical clock and therefore we cannot solve mutual exclusion problem using shared variables. To eliminate the mutual exclusion problem in distributed system approach based on message passing is used. A site in distributed system do not have complete information of state of the system due to lack of shared memory and a common physical clock.

Requirements of Mutual exclusion Algorithm:

- No Deadlock: Two or more site should notendlessly wait for any message that will never arrive.
- No Starvation: Every site who wants to execute critical section should get an opportunity to execute it in finite time. Any site should not wait indefinitely to execute critical section while other site is repeatedly executing critical section
- Fairness: Each site should get a fair chance to execute critical section. Any request to execute critical section must be executed in the order they are made i.e Critical section execution requests should be executed in the order of their arrival in the system.
- Fault Tolerance: In case of failure, it should be able to recognize it by itself in order to continue functioning without any disruption.

Solution to distributed mutual exclusion: As we know shared variables or a local kernel cannot be used to implement mutual exclusion in distributed systems. Message passing is a way to implement mutual exclusion. Below are the three approaches based on message passing to implement mutual exclusion in distributed systems:

1. Token Based Algorithm:

- A unique token is sharedamong all the sites.
- If a site possesses the unique token, it is allowed to enter its critical section
- This approach uses sequence number to order requests for the critical section.
- Each request for critical section contains a sequence number. This sequence number is used to distinguish old and current requests.

This approach insures Mutual exclusion as the token is unique

Example : Suzuki–Kasami Algorithm

2. Non-token-based approach:

- A site communicates with other sites in order to determine which sites should execute critical section next. This requires exchange of two or more successive round of messages among sites.
- This approach uses timestamps instead of sequence number to order requests for the critical section.

Whenever a site make request for critical section, it gets a timestamp. Timestamp is also used to resolve any conflict between critical section requests.

- All algorithm which follows non-token-based approach maintains a logical clock. Logical clocks get updated according to Lamport's scheme

Example : Ricart–Agrawala Algorithm

3. Quorum based approach:

- Instead of requesting permission to execute the critical section from all other sites, each site requests only a subset of sites which is called a quorum.
- Any two subsets of sites or Quorum contain a common site.
- This common site is responsible to ensure mutual exclusion

Example : Maekawa's Algorithm

```
from multiprocessing import Process, Value
from time import sleep

# Number of processes
n = 10

# Logical clocks for each process
clock = [Value('i', 0) for i in range(n)]

# Request flags for each process (1 if requesting, 0 otherwise)
request = [Value('i', 0) for i in range(n)]

def request_cs(pid):
    """ Function to request and enter the critical section """
    # Set request flag and increment logical clock
    request[pid].value = 1
    clock[pid].value += 1

    # Check other processes before entering critical section
    for j in range(n):
        if j != pid:
            while request[j].value == 1:  # If another process is requesting
                # Check timestamp priority (lower timestamp gets priority)
                if clock[j].value < clock[pid].value:
                    clock[pid].value += 1
                # If timestamps are the same, use process ID as tie-breaker
                elif (clock[j].value == clock[pid].value) and (j < pid):
                    clock[pid].value += 1
                else:
                    break  # Current process has priority, exit loop

    # Entering the critical section
    print(f"Process {pid} enters critical section at {clock[pid].value}")
    sleep(1)  # Simulate critical section work

    # Exiting the critical section
    print(f"Process {pid} exits critical section")
    request[pid].value = 0  # Reset request flag

if __name__ == "__main__":
    # Create process instances
    processes = [Process(target=request_cs, args=(i,)) for i in range(n)]

    # Start all processes
    for p in processes:
        p.start()

    # Wait for all processes to finish
    for p in processes:
        p.join()
```

```
PS C:\Users\LENOVO\OneDrive\Desktop\DC EXP> python app.py
Process 1 enters critical section at 1
Process 0 enters critical section at 1
Process 2 enters critical section at 1
Process 4 enters critical section at 1
Process 3 enters critical section at 1
Process 5 enters critical section at 1
Process 6 enters critical section at 1
Process 7 enters critical section at 1
Process 8 enters critical section at 1
Process 7 enters critical section at 1
Process 7 enters critical section at 1
Process 8 enters critical section at 1
Process 9 enters critical section at 1
Process 0 exits critical section
Process 7 enters critical section at 1
Process 8 enters critical section at 1
Process 9 enters critical section at 1
Process 0 exits critical section
Process 9 enters critical section at 1
Process 0 exits critical section
Process 0 exits critical section
Process 1 exits critical section
Process 2 exits critical section
Process 4 exits critical section
Process 2 exits critical section
Process 4 exits critical section
Process 4 exits critical section
Process 3 exits critical section
Process 3 exits critical section
Process 6 exits critical section
Process 6 exits critical section
Process 5 exits critical section
Process 5 exits critical section
Process 7 exits critical section
Process 7 exits critical section
Process 8 exits critical section
Process 8 exits critical section
Process 9 exits critical section
PS C:\Users\LENOVO\OneDrive\Desktop\DC EXP>
```

Conclusion:

This experiment successfully demonstrates Distributed Mutual Exclusion (DME) using logical clocks and request flags. It ensures only one process accesses the critical section at a time while maintaining fairness and preventing starvation. The approach effectively synchronizes processes, making it useful for real-world distributed systems like databases and cloud computing.

```python
from multiprocessing import Process, Value
from time import sleep

# Number of processes
n = 5 # Reduced from 10 for clearer output

# Shared logical clocks for each process ('i' stands for signed integer)
clock = [Value('i', 0) for _ in range(n)]

# Request flags for each process (1 if requesting, 0 otherwise)
request = [Value('i', 0) for _ in range(n)]

def request_cs(pid):
    """Function to request and enter the critical section"""

    # Set request flag and increment logical clock
    request[pid].value = 1
    clock[pid].value += 1

    print(f"Process {pid} is requesting CS with timestamp {clock[pid].value}")

    # Check other processes before entering critical section
    for j in range(n):
        if j != pid:
            while request[j].value == 1:
                # Check timestamp priority (lower timestamp gets priority)
                if clock[j].value < clock[pid].value:
                    # Wait/Yield to the other process
                    sleep(0.1)
                    continue
                # If timestamps are the same, use process ID as tie-breaker
                elif (clock[j].value == clock[pid].value) and (j < pid):
                    sleep(0.1)
                    continue
                else:
                    # Current process has higher priority
                    break

    # Entering the critical section
    print(f"--- Process {pid} ENTERS critical section (Time: {clock[pid].value}) ---")
    sleep(1) # Simulate critical section work

    # Exiting the critical section
    print(f"--- Process {pid} EXITS critical section ---")
    request[pid].value = 0 # Reset request flag

if __name__ == "__main__":
    # Create process instances
    processes = [Process(target=request_cs, args=(i,)) for i in range(n)]

    # Start all processes
    for p in processes:
        p.start()

    # Wait for all processes to finish
    for p in processes:
        p.join()

    print("All processes have finished.")
```